UNIVERSITY OF WESTMINSTER

5COSC019W · Object Oriented Programming

Tutorial 2 — Student Lab Guide

Encapsulation, the JSON Connection & UML Class Diagrams

Detail	Value
Module	5COSC019W — Object Oriented Programming
Level	Level 5 — 20 Credits, School of Computer Science and Engineering
Session	Tutorial 2 — 2-hour lab, university computer lab (IntelliJ via AppsAnywhere)
Builds on	Tutorial 1 (`products-api` project) and Lecture 2
Learning outcomes	LO1, LO2, LO5
Phase	Phase 1: Foundations (Lectures 1–3)

PURPOSE

This lab turns Lecture 2 into working code. You will build the Product class and its endpoint, watch Spring's JSON response break and heal as you change one keyword at a time, draw the UML class diagram, and finish by designing a Customer class that contains an Address. Everything you write today is committed to the same GitHub repository you created in Tutorial 1.

HOW TO USE THIS GUIDE

Steps are numbered in one continuous sequence from 1 to 49. Do them in order — each one assumes the one before it worked.

Text in `this font` is typed exactly as shown, including capital letters, brackets and semicolons.

An arrow like Settings → Version Control → GitHub means: click each of those in turn.

Every activity ends with a CHECKPOINT. Do not move on until yours passes.

Stuck for more than five minutes? Raise your hand. That is what the session is for.

IMPORTANT — YOU ARE ON A SHARED, NON-PERSISTENT MACHINE

This lab runs on a university PC, not your own laptop, and IntelliJ IDEA is streamed through AppsAnywhere. Two things follow from that:

1. Local files are not guaranteed to survive after you log out. GitHub is your real save button — commit and push after every activity, not just at the end.

2. Someone else uses this machine after you. Sign out of GitHub and IntelliJ before you leave (see the Wrap-Up).

Before You Start

Have these ready. There is no time in a 2-hour lab to sort them out from scratch.

- Your Tutorial 1 GitHub repository — you will add to it today, not start a new one.
- Your GitHub username and password (you log in through a browser window).
- Your university username and password for the lab PC and AppsAnywhere.

- Ten minutes spent re-reading the Lecture 2 slides on getters and the JavaBean naming convention. The lab assumes you have seen them.

What You Are Building Today

At the end of Tutorial 1 your project looked like this:

```
products-api
└── src/main/java/uk/ac/westminster/productsapi
    ├── ProductsApiApplication.java
    ├── HelloController.java
    └── Person.java
```

By the end of today it will look like this:

```
products-api
├── docs/uml/                        <-- your diagrams (Activities 4 and 6)
└── src/main/java/uk/ac/westminster/productsapi
    ├── ProductsApiApplication.java
    ├── HelloController.java
    ├── Person.java
    ├── Product.java                 <-- new today
    ├── ProductController.java       <-- new today
    ├── Address.java                 <-- new today
    ├── Customer.java                <-- new today
    └── CustomerController.java      <-- new today
```

NOTE

Tutorial 1 never created a Product class or a product endpoint — the project only has `Person` and `HelloController`. You build Product and its endpoint from scratch in Activity 1, so nothing is missing if you cannot find them.

Session Timing at a Glance

Time	Activity
0:00 – 0:10	Recap & Environment Check — confirm your Tutorial 1 project still runs
0:10 – 0:30	Activity 1 — Create the Product class and its /products endpoint
0:30 – 0:50	Activity 2 — Encapsulate Product: private fields, getters, constructors
0:50 – 1:05	Activity 3 — Break it, live: remove one getter and watch the JSON change
1:05 – 1:25	Activity 4 — Draw the Product UML class diagram (any tool you like)
1:25 – 1:45	Activity 5 — Build Customer with a nested Address
1:45 – 1:55	Activity 6 — UML for Customer and Address; final commit and push
1:55 – 2:00	Wrap-Up — submit your link, sign out of everything, log off

Learning Objectives for This Lab

- Create a Java class and a Spring REST controller from scratch, in the correct package.
- Change a field from public to private and predict, then observe, the effect on the JSON response.
- Write getters that follow the JavaBean naming convention, and explain why Spring needs them.

- Draw a correct three-compartment UML class diagram using visibility symbols.
- Model an association between two classes (Customer has-an Address) and see it as nested JSON.
- Commit and push incrementally with meaningful messages.

Recap & Environment Check (10 min)

GOAL

Confirm the project you built in Tutorial 1 still runs exactly as you left it. This is diagnostic: if something is broken, find out now rather than halfway through Activity 2.

#	What to do	How you know it worked
1	Log in to the lab PC, open AppsAnywhere, and launch Git, then IntelliJ IDEA. First launch can take a few minutes while the IDE streams — be patient.	IntelliJ opens.
2	Open your `products-api` project. If it is not on this machine, clone it: Welcome screen → Get from VCS → paste your GitHub repository URL → Clone.	The project opens and the Project pane on the left shows `src`.
3	Wait for Maven to finish loading (progress bar along the bottom). If `@RestController` shows red: right-click `pom.xml` → Maven → Sync Project.	No red underlines in `HelloController.java`.
4	Open `ProductsApiApplication.java` and click the green Run arrow (or press Shift + F10).	The console shows `Started ProductsApiApplication`.
5	In a browser, open `http://localhost:8080/swagger-ui.html`. Check that `/hello`, `/goodbye` and `/status` are all listed.	Swagger UI loads and all three endpoints appear.

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

Your Tutorial 1 application is running and Swagger UI lists three endpoints. If not, get help now — every later activity depends on this.

TROUBLESHOOTING — RECAP & ENVIRONMENT CHECK

Port 8080 already in use: an old copy of the app is still running. Click the red square in the Run panel to stop it, then run again.

Swagger UI shows a white page or 404: check the app is actually running (console says `Started`), and that you typed `swagger-ui.html`, not `swagger-ui`.

Project will not open: ask a tutor rather than starting a new project — you need your existing Git history.

Activity 1 — Create the Product Class and Its Endpoint (20 min)

GOAL

Lecture 1 showed you the Product class, but you never added it to the Spring project — and no endpoint has ever returned an object instead of a String. You build both now. Write the fields as public for the moment; Activity 2 explains why that is a bad idea.

#	What to do	How you know it worked
6	In the Project pane, expand `products-api` → `src` → `main` → `java` → `uk.ac.westminster.productsapi`. This is the package where `ProductsApiApplication.java` lives.	You can see `ProductsApiApplication.java`, `HelloController.java` and `Person.java` together.
7	Right-click the package name `uk.ac.westminster.productsapi` → New → Java Class. Type `Product` and press Enter. Do not put it in a new folder — it must sit beside the other files.	`Product.java` opens, already containing the correct `package` line.
8	Type the Product class from Code Reference 1 below. Type it rather than pasting — you will remember it better, and typos teach you to read compiler messages.	`Product.java` compiles: no red underlines.
9	Right-click the same package → New → Java Class → type `ProductController` → Enter. Type the code from Code Reference 2 below, including the four `import` lines.	`ProductController.java` shows no red underlines.
10	Stop the application (red square), then start it again (green arrow). Spring Boot does not pick up new code while it is running.	Console shows `Started ProductsApiApplication` again.
11	Refresh Swagger UI in the browser. Look for a new section called `product-controller`.	`GET /products/{id}` is listed under `product-controller`.
12	Click `GET /products/{id}` → Try it out → type `1` in the id box → Execute. Read the Response body.	The response body is: `{ "id": 1, "name": "Laptop", "price": 999.99 }` and the status code is `200`.
13	Commit your work: Git → Commit, tick the new files, message: `Tutorial 2: Product class and /products endpoint added` then click Commit and Push.	The two new files are visible on GitHub in your browser.

Code Reference 1 — Product.java (first version, public fields)

```
package uk.ac.westminster.productsapi;

public class Product {

    public Long id;
    public String name;
    public double price;

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
}
```

Code Reference 2 — ProductController.java

```
package uk.ac.westminster.productsapi;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public Product getById(@PathVariable Long id) {
        return new Product(id, "Laptop", 999.99);
    }
}
```

WHY PUBLIC FIELDS, JUST FOR NOW?

In Lecture 1 the Product fields had no access modifier at all. Written that way, Spring's JSON converter cannot read them and the response comes back empty — which would spoil the experiment.

Writing them `public` gives you a version that works, so that when it stops working in Activity 2 you know exactly which change caused it. You will fix the design in twenty minutes.

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

Asking for `/products/1` in Swagger UI returns three fields of JSON. You have just made a controller method return an object instead of a String — the exact change Lecture 2 opened with.

TROUBLESHOOTING — ACTIVITY 1 — CREATE THE PRODUCT CLASS AND ITS ENDPOINT

404 or a Whitelabel Error Page at /products/1: `ProductController.java` is probably in the wrong package. It must be in `uk.ac.westminster.productsapi`, the same package as `ProductsApiApplication`. Drag it there in the Project pane and restart.

`Cannot resolve symbol 'Product'`**: the class name inside the file must match the file name exactly, including the capital P.

Red underlines on `@RestController` or `@GetMapping`: the `import` lines are missing. Put the cursor on the red word and press Alt + Enter → Import class.

The new endpoint does not appear in Swagger UI: you did not restart. Spring Boot never reloads code by itself in this project.

Activity 2 — Encapsulate Product (20 min)

GOAL

Turn Product into a properly encapsulated class: private fields, getters, and both constructors. Along the way you will see the empty-JSON failure from the lecture on your own machine.

#	What to do	How you know it worked
14	In `Product.java`, change all three fields from `public` to `private`. Change nothing else yet. Do not add getters.	The file still compiles — no red underlines anywhere.
15	Before you run it, write down your prediction: what will `/products/1` return now? Say it out loud to the person next to you.	You have committed to an answer before seeing the result.
16	Restart the application. In Swagger UI, run `GET /products/{id}` with id `1` again.	The response body is `{ }` — completely empty. The status code is still `200`. There is no error message anywhere.
17	Look at the IntelliJ console. Scroll to the bottom. Confirm for yourself that nothing was logged — no exception, no warning.	The console shows normal start-up output and nothing else.
18	Now add the three getters from Code Reference 3 below. Method names matter: `getId`, `getName`, `getPrice` — capital letter after `get`.	The class compiles with three new methods.
19	Add the no-arg constructor as well: `public Product() {}` Keep the full constructor too. A class can have more than one.	`Product.java` matches Code Reference 3 exactly.
20	Restart and run `GET /products/{id}` with id `1` once more.	All three fields are back: `{ "id": 1, "name": "Laptop", "price": 999.99 }`
21	Commit: `Tutorial 2: Product encapsulated with private fields and getters`	The commit appears in Git → Show Git Log.

Code Reference 3 — Product.java (encapsulated)

```
package uk.ac.westminster.productsapi;

public class Product {

    private Long id;
    private String name;
    private double price;

    public Product() {}

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public Long getId() { return id; }

    public String getName() { return name; }

    public double getPrice() { return price; }
}
```

WHY ADD A CONSTRUCTOR THAT DOES NOTHING?

`public Product() {}` is not needed for reading data, which is all you do today. It is needed when JSON arrives the other way — in the body of a POST request — because Jackson creates an empty object first and fills the fields afterwards.

Add it now so the endpoint you write in Lecture 3 does not fail with a confusing HTTP 400 for a reason you have forgotten about.

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

You have seen the same endpoint return full JSON, then `{ }`, then full JSON again — with no error at any point. The only thing that changed was how the outside world is allowed to read the data.

TROUBLESHOOTING — ACTIVITY 2 — ENCAPSULATE PRODUCT

The JSON is still empty after adding getters: check the spelling. `getname()` and `GetName()` do not count — it must be `getName()`. Also check you restarted.

`Cannot resolve method 'getName'` somewhere else in the project**: something outside Product was reading the field directly. That is exactly what private prevents — change that code to call the getter instead.

Two constructors look like an error: they are not. Java allows several constructors as long as their parameter lists differ.

Activity 3 — Break It, Live (15 min)

GOAL

Reproduce the lecture demonstration one field at a time. This is the moment encapsulation clicks — do not skip it, even though it feels destructive.

#	What to do	How you know it worked
22	In `Product.java`, select the whole `getName()` method and press Ctrl + / to comment it out. The line turns grey and starts with `//`.	The class still compiles. Java does not mind a missing getter.
23	Predict first, then restart and run `GET /products/{id}` with id `1`.	The response is `{ "id": 1, "price": 999.99 }` — `name` has vanished. Status is still `200`. No error.
24	Explain to the person next to you, in your own words, why `name` disappeared but `id` and `price` did not.	You can say something like: Jackson looks for a public `getX()` method for each field; with no `getName()` there is nothing for it to call, so the field is skipped.
25	Now imagine this happened in your coursework project, in a class with fifteen fields. Write one sentence in your notes about how you would notice.	You have a written answer — this is the kind of bug that produces no error message.
26	Select the commented method and press Ctrl + / again to restore it. Restart and confirm all three fields return.	The full JSON is back.
27	Commit and push: `Tutorial 2: verified getter removal drops the field from JSON`	Your GitHub repository shows the commit.

KEY POINT

No compile error. No exception. No warning in the console. A `200 OK` status with part of the data missing.

This is the single most common way a Spring API quietly returns the wrong thing, and the reason encapsulation is a design decision rather than a style preference.

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

You have made a field silently disappear from an API response and put it back, and you can explain why in one sentence without looking at your notes.

Activity 4 — Draw the Product UML Class Diagram (20 min)

GOAL

Capture the class you just wrote in professional notation. Draw it by hand first — the aim is fluency with the notation, not a tidy picture. Then reproduce it in whichever tool you prefer.

#	What to do	How you know it worked
28	On paper, draw a rectangle and divide it into three stacked compartments. Write `Product` in the top compartment.	Three compartments, name at the top.

#	What to do	How you know it worked
29	In the middle compartment, list the three attributes in the form `visibility name : Type`. All three fields are private, so all three start with `-`.	`- id : Long`, `- name : String`, `- price : double`
30	In the bottom compartment, list both constructors and the three getters. All are public, so all start with `+`. Constructors have no return type.	Five entries, each beginning with `+`.
31	Check your diagram against the UML checklist in Appendix C. Fix anything that does not match before going digital.	Every line of the checklist ticks.
32	Choose any tool from Appendix B and reproduce the diagram. draw.io, PlantUML, Mermaid, PowerPoint shapes, or a clear photo of your paper version are all acceptable.	You have a file, not just a sketch on a desk.
33	In IntelliJ, right-click the project root `products-api` → New → Directory → type `docs/uml` → Enter. Save or export your diagram into that folder as `Product` with whatever extension your tool produces.	`docs/uml/Product.*` appears in the Project pane.
34	Commit and push: `Tutorial 2: Product UML class diagram added`	The diagram file is visible on GitHub.

USE WHICHEVER TOOL YOU LIKE

There is no required UML tool on this module. The marks are for correct notation, not for the software that produced it.

A clearly legible photograph of a hand-drawn diagram is worth exactly as much as a generated one — as long as the notation is right and the image is readable on screen.

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

A three-compartment Product diagram exists in your repository, with `-` on every attribute and `+` on every method.

Activity 5 — Build Customer with a Nested Address (20 min)

GOAL

Apply the same encapsulation pattern to two new classes, and create your first association: a Customer has-an Address. You will see the relationship appear in the JSON as a nested object.

#	What to do	How you know it worked
35	In the same package, create a class `Address` (right-click package → New → Java Class). Type the code from Code Reference 4.	`Address.java` compiles: three private fields, two constructors, three getters.
36	Create a class `Customer` from Code Reference 5. Notice the fourth field: its type is `Address`, not `String`. One class can hold another.	`Customer.java` compiles and refers to `Address`.

#	What to do	How you know it worked
37	Create a class `CustomerController` from Code Reference 6. It builds an Address first, then passes it into the Customer constructor.	No red underlines.
38	Restart the application and refresh Swagger UI.	A new `customer-controller` section appears with `GET /customers/{id}`.
39	Run `GET /customers/{id}` with id `1` → Execute. Look carefully at the shape of the response, not just the values.	The address is a nested object inside the customer, with its own pair of braces — see the Expected Response box below.
40	Experiment: comment out `getAddress()` in `Customer.java` with Ctrl + /, restart, and run the endpoint again.	The entire address block disappears — all three of its fields at once.
41	Restore `getAddress()`, restart, confirm the address is back, then commit: `Tutorial 2: Customer and Address classes with association`	Full nested JSON returns and the commit is pushed.

Code Reference 4 — Address.java

```
package uk.ac.westminster.productsapi;

public class Address {

    private String street;
    private String city;
    private String postcode;

    public Address() {}

    public Address(String street, String city, String postcode) {
        this.street = street;
        this.city = city;
        this.postcode = postcode;
    }

    public String getStreet() { return street; }

    public String getCity() { return city; }

    public String getPostcode() { return postcode; }
}
```

Code Reference 5 — Customer.java

```
package uk.ac.westminster.productsapi;

public class Customer {

    private Long id;
    private String name;
    private String email;
    private Address address;

    public Customer() {}

    public Customer(Long id, String name, String email, Address address) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.address = address;
    }

    public Long getId() { return id; }

    public String getName() { return name; }

    public String getEmail() { return email; }

    public Address getAddress() { return address; }
}
```

Code Reference 6 — CustomerController.java

```
package uk.ac.westminster.productsapi;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/customers")
public class CustomerController {

    @GetMapping("/{id}")
    public Customer getById(@PathVariable Long id) {

        Address address = new Address(
                "115 New Cavendish Street", "London", "W1W 6UW");

        return new Customer(id, "Ada Lovelace",
                "ada@example.com", address);
    }
}
```

Expected Response — GET /customers/1

```
{
  "id": 1,
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "address": {
    "street": "115 New Cavendish Street",
    "city": "London",
    "postcode": "W1W 6UW"
  }
}
```

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

`/customers/1` returns a customer with an address nested inside it, and you have seen that one missing getter can remove a whole object from the response.

TROUBLESHOOTING — ACTIVITY 5 — BUILD CUSTOMER WITH A NESTED ADDRESS

`Cannot resolve symbol 'Address'`**: `Address.java` is in a different package, or the file was created before you typed the class name. Check it sits beside `Customer.java`.

The address shows as `null`: the controller passed nothing into the constructor. Compare your controller line by line with Code Reference 6.

Only some address fields appear: one of Address's getters is missing or misspelled.

Activity 6 — UML for Customer and Address, Final Commit & Push (10 min)

GOAL

Record the design you just implemented, including the relationship between the two classes, and leave the session with everything pushed.

#	What to do	How you know it worked
42	Draw a three-compartment box for `Customer` and another for `Address`, side by side. Use the same rules as Activity 4: `-` on attributes, `+` on methods.	Both boxes complete, including `- address : Address` inside Customer.
43	Draw a solid line from Customer to Address with an open arrowhead pointing at Address. This is an association: it reads as 'a Customer has an Address'.	One line, arrow pointing at Address, not the other way round.
44	Reproduce it in your chosen tool and save it as `docs/uml/CustomerAddress.*`	The file appears in the Project pane.
45	Commit everything: `Tutorial 2: encapsulation and UML for Product, Customer and Address` Then Git → Push.	Push succeeds with no error dialog.
46	Open your repository on github.com in the browser and check all six new files are there: `Product.java`, `ProductController.java`, `Address.java`, `Customer.java`, `CustomerController.java`, and your two diagrams.	Everything is visible on GitHub, not just on this PC.

RUNNING OUT OF TIME?

Push whatever you have before 1:55 — a partial commit is far better than nothing, and the history is part of your LO5 evidence.

Finish the remaining diagram at home and push it before the end of the week. Add a commit message that says what you completed later, for example `Tutorial 2: completed Customer/Address UML after the session`.

CHECKPOINT — DO NOT MOVE ON UNTIL THIS IS TRUE

Your GitHub repository contains today's five Java files, two UML diagrams, and at least four commits made during this session.

Wrap-Up (5 min) — Read This Before You Leave

These three steps are not optional. The first is your deliverable; the other two protect your accounts on a shared machine.

#	What to do	How you know it worked
47	Paste your GitHub repository URL into the Blackboard submission link for Tutorial 2.	Submission recorded before you leave the room.
48	Sign out of GitHub in the browser: click your avatar (top right) → Sign out. In IntelliJ: Settings → Version Control → GitHub → select your account → remove it.	No account of yours is left signed in on this machine.
49	Close IntelliJ and log off the university PC completely.	The next student sees a clean login screen.

WHY THE COMMIT HISTORY MATTERS

This repository is part of the 60% coursework. A repo with one commit on submission day looks very different from one showing eleven weeks of real work.

Today should add at least four commits. That pattern, repeated weekly, is what evidences LO5.

Optional — Stretch Tasks

For anyone who finishes early. Not collected, not marked — practice.

- Add a `static` field `customerCount` to Customer, increment it in both constructors, and expose it with `public static int getCustomerCount()`. Show it underlined in your UML.
- Add a plain array field to Customer, for example `String[] tags`, following the same encapsulation pattern. Use an array rather than a List — lists arrive properly in Lecture 3.
- Remove the `@RequestMapping("/products")` line from ProductController and change the method's mapping to `@GetMapping("/products/{id}")`. Confirm the endpoint still works, and explain to a neighbour what `@RequestMapping` was doing for you.

Appendix A — Choosing a UML Tool

There is no required UML tool on this module, and no tool earns you marks by itself. Pick whichever of these you will actually use, and spend your time on getting the notation right.

By hand, then photographed

Draw it on paper, photograph it with your phone in good light, and drag the image into `docs/uml`. Perfectly acceptable — make sure every symbol is readable when the image is viewed on a laptop screen.

draw.io (diagrams.net)

Open app.diagrams.net in a browser, choose a blank diagram, and use Shape Search for "UML class". Type the compartments directly into the shape. File → Export as → PNG, or File → Save as → .drawio to keep it editable.

PlantUML

Go to plantuml.com and use the online editor, or install the PlantUML plugin in IntelliJ (Settings → Plugins). You write text and it renders the diagram. Source for both of today's diagrams is given below.

Mermaid

Use mermaid.live in a browser, or a Mermaid code block inside a Markdown file — GitHub renders those automatically, so a `.md` file in `docs/uml` will display as a diagram in your repository.

PowerPoint, Word or Google Slides

Rectangles plus horizontal lines are enough for a class box, and a straight-line connector with an arrowhead is enough for an association. Export the slide as PNG or PDF.

PlantUML source — all three classes (one option, not a requirement)

```
@startuml

class Product {
  - id : Long
  - name : String
  - price : double
  + Product()
  + Product(id : Long, name : String, price : double)
  + getId() : Long
  + getName() : String
  + getPrice() : double
}

class Customer {
  - id : Long
  - name : String
  - email : String
  - address : Address
  + Customer()
  + Customer(id : Long, name : String, email : String, address : Address)
  + getId() : Long
  + getName() : String
  + getEmail() : String
  + getAddress() : Address
}

class Address {
  - street : String
  - city : String
  - postcode : String
  + Address()
  + Address(street : String, city : String, postcode : String)
  + getStreet() : String
  + getCity() : String
  + getPostcode() : String
}

Customer --> Address

@enduml
```

Mermaid source — Customer and Address (renders directly on GitHub)

```
classDiagram
    class Customer {
        -Long id
        -String name
        -String email
        -Address address
        +getId() Long
        +getName() String
        +getEmail() String
        +getAddress() Address
    }
    class Address {
        -String street
        -String city
        -String postcode
        +getStreet() String
        +getCity() String
        +getPostcode() String
    }
    Customer --> Address
```

Appendix B — UML Checklist

Check your diagram against every line before you commit it. This is what a marker looks for.

- The box has exactly three compartments, in this order: class name, attributes, methods.
- The class name is spelled exactly as in your Java file, with a capital first letter.
- Every attribute is written `visibility name : Type` — the type comes after the colon.
- Every private field is marked `-`. Nothing today should be marked `+` in the attributes compartment.
- Every public method is marked `+`.
- Constructors are listed like methods but with no return type.
- Getter return types match the field types exactly: `getPrice() : double`, not `: Double`.
- For Customer and Address: a solid line connects them, with the arrow pointing at Address.
- Any static member is underlined (nothing today unless you attempt the stretch task).

Appendix C — Error Messages and What They Mean

What you see	What to do
Whitelabel Error Page, status 404	The URL does not match any mapping, or the controller is in the wrong package. Check spelling and that the class sits in `uk.ac.westminster.productsapi`.
Web server failed to start. Port 8080 was already in use.	An earlier run is still going. Stop it with the red square, or restart IntelliJ.
Response body is `{ }`	The class has no public fields and no getters. Expected in Activity 2 step 15 — a bug at any other time.
One field missing from the JSON	Its getter is missing or misspelled. `getname()` is not the same as `getName()`.
`Cannot resolve symbol` on a class name	The file name and the class name must match exactly, and both files must be in the same package.
Red underline on an annotation	Missing import. Put the cursor on it and press Alt + Enter → Import class.
Push rejected / authentication failed	Re-add your GitHub account: Settings → Version Control → GitHub → Add account → Log In with GitHub.

Appendix D — Recommended Resources

- Lecture 2 slides — particularly Sections 02 and 05 (Blackboard)
- Baeldung — Access Modifiers in Java: baeldung.com/java-access-modifiers
- Baeldung — Significance of Getters and Setters in Java: baeldung.com/java-why-getters-setters
- Oracle Docs — Controlling Access to Members of a Class
- UML tools, your choice: diagrams.net · plantuml.com · mermaid.live · PowerPoint · paper

Assessment Alignment

Outcome	Tutorial 2 evidence	Contributes to
LO1	Product, Customer and Address implemented with private fields, constructors and getters; observed effect of access modifiers on program behaviour	Enterprise Application + Lab Practical
LO2	A REST endpoint returning an object rather than a String; `@RequestMapping` and `@PathVariable` used; responses verified in Swagger UI	Enterprise Application
LO5	UML class diagrams for Product and Customer/Address, committed with meaningful messages across the session	Enterprise Application

SUBMISSION ACTION

Paste your GitHub repository URL into the Blackboard submission link before you leave. This week's UML diagrams and encapsulated classes are part of your ongoing LO5 evidence trail.

STUCK AFTER THE SESSION?

A model solution for Product, Customer and Address will be released on Blackboard a few days after this lab, once most students have submitted.

Use it to check your own work. Your class and variable names do not need to match it exactly.